

实验三：聊天记录压缩与信息提取

实验三：聊天记录压缩与信息提取

实验目的

1. 理解聊天历史管理的重要性和挑战
2. 掌握聊天记录压缩策略和实现方法
3. 学习关键信息提取技术（5W 规则）
4. 实现聊天历史搜索功能

实验内容

实验过程

1. 代码结构分析 本实验包含两个 Python 文件：

chat_client_with_summary.py - 聊天记录压缩 - `calculate_context_length()`: 计算上下文总字符长度 - `count_chat_rounds()`: 计算对话轮次 - `should_compress()`: 判断是否需要压缩 - `summarize_chat_history()`: 使用 LLM 总结聊天记录

chat_client_with_extraction.py - 信息提取与搜索 - `extract_key_information()`: 按 5W 规则提取关键信息 - `save_key_information()`: 保存提取的信息到日志 - `is_search_request()`: 判断是否为搜索请求 - `search_chat_history()`: 搜索聊天历史

2. 聊天记录压缩机制 压缩触发条件

```
def should_compress(chat_history, max_rounds=5, max_length=3000):  
    rounds = count_chat_rounds(chat_history)  
    length = calculate_context_length(chat_history)  
    return rounds >= max_rounds or length >= max_length
```

压缩策略 - 当对话超过 5 轮或上下文超过 3000 字符时触发压缩 - 前 70% 的对话内容进行总结压缩 - 后 30% 的对话内容保留原文 - 保持对话的连贯性和上下文理解

总结提示词

```
summary_prompt = f"""
```

请对以下对话历史进行简明扼要的总结：

```
{json.dumps(messages_to_summarize, ensure_ascii=False, indent=2)}
```

请用中文进行总结，保持关键信息完整，语言简洁。

```
"""
```

3. 关键信息提取（5W 规则） 5W 规则定义 - Who（谁）：参与对话的人物或角色 - What（做什么事）：对话中讨论的主要事件或内容 - When（什么时候）：提到的时间 - Where（在何处）：提到的地点 - Why（为什么）：做这件事的原因

信息提取实现

```
def extract_key_information(base_url, api_key, model, chat_history):
```

```
    extraction_prompt = f"""
```

请从以下对话历史中提取关键信息，按照 5W 规则进行提取：

对话历史：

```
{json.dumps(conversation, ensure_ascii=False, indent=2)}
```

请按照以下格式输出：

- Who（谁）：

- What（做什么事）：

- When（什么时候）：

- Where（在何处）：

- Why（为什么）：

```
"""
```

4. 聊天历史搜索 搜索触发 - 使用 /search 命令开头 - 包含“查找聊天历史”、“搜索历史记录”等关键词

搜索实现

```
def search_chat_history(base_url, api_key, model, user_query):
```

```
    log_content = load_log_file()
```

```
    if not log_content:
```

```
        return "暂无聊天历史记录可供搜索。"
```

```
    search_prompt = f"""
```

请根据以下日志内容和用户查询，提供相关的回答：

日志内容：

```
{log_content}
```

```
用户查询：
{user_query}
"""
```

实验结果

功能验证

功能	chat_client_with_summary.py	chat_client_with_extraction.py
聊天记录压缩	正确	正确
关键信息提取	错误	正确
信息保存到日志	错误	正确
聊天历史搜索	错误	正确
网络访问工具	正确	正确

输出示例 压缩功能输出

```
=====
正在执行聊天记录压缩...
总对话轮次：8
需要总结的消息数：5
保留的消息数：3
=====

总结内容：
用户询问了Python编程问题，包括列表操作和字典使用方法，我提供了相应的代码示例和解释。

=====
聊天记录已成功压缩！
```

信息提取输出

```
=====
正在提取关键信息...
=====

提取的关键信息：
- Who（谁）：用户、AI助手
- What（做什么事）：用户询问Python编程问题，包括列表操作和字典使用
- When（什么时候）：2024年1月15日
- Where（在何处）：无
- Why（为什么）：用户学习Python编程
```

=====

关键信息已保存到：D:\chat-log\log.txt

实验总结

实验成果

1. 实现了智能聊天记录压缩功能，有效控制上下文长度
2. 实现了基于 5W 规则的关键信息提取，便于信息归档
3. 实现了聊天历史搜索功能，支持自然语言查询
4. 建立了日志记录机制，持久化存储关键信息

技术要点

1. **压缩算法**: 采用 70/30 分割策略，平衡信息保留和压缩效果
2. **信息提取**: 使用 5W 规则结构化提取关键信息
3. **搜索机制**: 基于 LLM 的语义搜索，支持自然语言查询
4. **自动触发**: 根据对话轮次和长度自动触发压缩和提取

改进建议

1. 可添加压缩策略配置，允许用户自定义阈值
2. 可实现多种压缩算法对比
3. 可添加搜索结果排序功能
4. 可实现信息可视化展示